BRACT's

Vishwakarma Institute of Information Technology, Pune Practical Implementation Sheet

Department: CS-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No:4 Roll no: 21 PRN No: 12411682

Subject Name: Deep Learning

Name of Student: Pranay Zunzunwala

Problem Statement: Develop an LSTM-based model for time-series forecasting using stock price, weather, or sales datasets

Algorithm:

Step 1: Import the required libraries such as NumPy, Pandas, Matplotlib, TensorFlow/Keras, YFinance, and the necessary modules from scikit-learn.

Step 2: Load the **Apple (AAPL) stock dataset** using Yahoo Finance for the specified date range and inspect the dataset by displaying its shape, columns, and sample records.

Step 3: Perform **data preprocessing** by handling missing values, selecting the required stock-price feature, and arranging the data in chronological order.

Step 4: Normalize the selected stock-price data using **MinMaxScaler** to scale the values to a suitable range for training the neural network.

Step 5: Create **time-series sequences** using a fixed number of previous observations as input and the next stock-price value as the target.

Step 6: Split the generated sequences into **training and testing datasets** while maintaining the chronological order of the stock data.

Step 7: Build an **LSTM-based Sequential Neural Network** using LSTM, Dropout, and Dense layers, and compile the model using the Adam optimizer and an appropriate loss function.

Step 8: Train the LSTM model on the training data using a specified number of **epochs and batch size**, while using early stopping to prevent overfitting.

Step 9: Use the trained model to **predict stock prices** for the testing dataset and evaluate its performance using **Mean Absolute Error (MAE)** and **Mean Squared Error (MSE/RMSE)**.

Step 10: Visualize the results by plotting the **actual vs. predicted AAPL stock prices** and analyze the model's prediction performance.

Code :

```
# Install required packages

!pip -q install yfinance tensorflow scikit-learn pandas matplotlib


import os

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import yfinance as yf

import tensorflow as tf

from sklearn.preprocessing import MinMaxScaler

from sklearn.metrics import mean_absolute_error, mean_squared_error

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import LSTM, Dense, Dropout

from tensorflow.keras.callbacks import EarlyStopping

print("TensorFlow version:", tf.__version__)

print("NumPy version:", np.__version__)

# Choose data source:

# "yahoo" = automatically download AAPL data

# "upload" = upload your own CSV file
```

```
DATA_SOURCE = "yahoo"
# Date range for the project
START_DATE = "2015-01-01"
END_DATE = "2026-01-01"
if DATA_SOURCE.lower() == "yahoo":
    df = yf.download(
        "AAPL",
        start=START_DATE,
        end=END_DATE,
        auto_adjust=False,
        progress=False
    )
    # Flatten MultiIndex columns if returned by yfinance
    if isinstance(df.columns, pd.MultiIndex):
        df.columns = df.columns.get_level_values(0)
    df = df.reset_index()
else:
    from google.colab import files
    uploaded = files.upload()
    filename = next(iter(uploaded))
    df = pd.read_csv(filename)
print("Shape:", df.shape)
display(df.head())
# Standardize column names and identify required columns
df.columns = [str(c).strip() for c in df.columns]
# Find Date and Close columns case-insensitively
```

```
date_col = next((c for c in df.columns if c.lower() == "date"), None)
close_col = next((c for c in df.columns if c.lower() == "close"), None)
if date_col is None or close_col is None:
    raise ValueError(
        f"Dataset must contain Date and Close columns. Found: {list(df.columns)}"
    )
# Keep useful columns if available
keep_cols = [date_col, close_col]
for c in ["Open", "High", "Low", "Adj Close", "Volume"]:
    if c in df.columns and c not in keep_cols:
        keep_cols.append(c)
df = df[keep_cols].copy()
df = df.rename(columns={date_col: "Date", close_col: "Close"})
# Convert types
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
df["Close"] = pd.to_numeric(df["Close"], errors="coerce")
# Clean
df = (
    df.dropna(subset=["Date", "Close"])
      .drop_duplicates(subset=["Date"])
      .sort_values("Date")
      .reset_index(drop=True)
)
print("Cleaned shape:", df.shape)
print("Date range:", df["Date"].min().date(), "to", df["Date"].max().date())
print("Missing values:")
```

```
print(df.isna().sum())
display(df.head())
display(df.tail())
plt.figure(figsize=(14, 5))
plt.plot(df["Date"], df["Close"])
plt.title("Apple (AAPL) Historical Closing Price")
plt.xlabel("Date")
plt.ylabel("Closing Price (USD)")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
# Basic statistics
display(df["Close"].describe().to_frame("Close Price"))
# Recent prices
display(df.tail(10)[["Date", "Close"]])
TRAIN_RATIO = 0.80
close_values = df[["Close"]].values.astype("float32")
split_idx = int(len(close_values) * TRAIN_RATIO)
train_raw = close_values[:split_idx]
test_raw = close_values[split_idx:]
train_dates = df["Date"].iloc[:split_idx].reset_index(drop=True)
test_dates = df["Date"].iloc[split_idx:].reset_index(drop=True)
print("Training observations:", len(train_raw))
print("Testing observations :", len(test_raw))
print("Training period:", train_dates.iloc[0].date(), "to", train_dates.iloc[-1].date())
print("Testing period :", test_dates.iloc[0].date(), "to", test_dates.iloc[-1].date())
```

```
scaler = MinMaxScaler(feature_range=(0, 1))
train_scaled = scaler.fit_transform(train_raw)
# Transform test data using the training-fitted scaler
test_scaled = scaler.transform(test_raw)
print("Original training range:", train_raw.min(), "to", train_raw.max())
print("Scaled training range  :", train_scaled.min(), "to", train_scaled.max())
LOOKBACK = 60
def create_sequences(data, lookback):
    X, y = [], []
    for i in range(lookback, len(data)):
        X.append(data[i - lookback:i, 0])
        y.append(data[i, 0])
    X = np.array(X, dtype=np.float32)
    y = np.array(y, dtype=np.float32)
    return X.reshape(-1, lookback, 1), y
X_train, y_train = create_sequences(train_scaled, LOOKBACK)
# For the test set, prepend the final 60 training values.
# This allows the first test prediction to use the immediately preceding history.
test_input = np.concatenate([train_scaled[-LOOKBACK:], test_scaled], axis=0)
X_test, y_test = create_sequences(test_input, LOOKBACK)
print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape :", X_test.shape)
print("y_test shape :", y_test.shape)
```

```
tf.keras.backend.clear_session()
model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(LOOKBACK, 1)),
    Dropout(0.20),
    LSTM(32),
    Dropout(0.20),

    Dense(16, activation="relu"),
    Dense(1)
])
model.compile(
    optimizer="adam",
    loss="mean_squared_error",
    metrics=["mae"]
)
model.summary()
early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=10,
    restore_best_weights=True
)
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
```

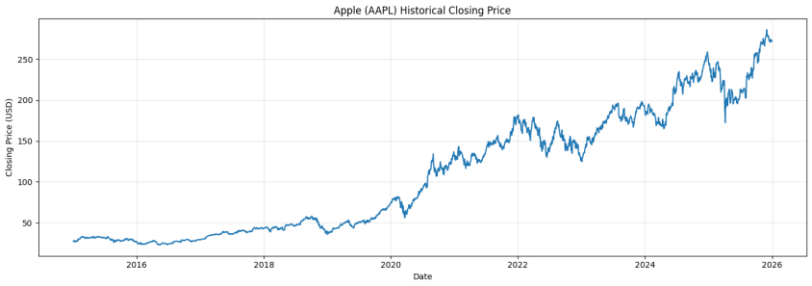
```
    validation_split=0.10,
    shuffle=False,
    callbacks=[early_stopping],
    verbose=1
)
# Plot training and validation loss
plt.figure(figsize=(10, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("LSTM Training History")
plt.xlabel("Epoch")
plt.ylabel("Mean Squared Error Loss")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
pred_scaled = model.predict(X_test, verbose=0)
# Convert predictions and actual values back to USD
predicted_prices = scaler.inverse_transform(pred_scaled).flatten()
actual_prices = scaler.inverse_transform(y_test.reshape(-1, 1)).flatten()
# Dates corresponding to the test predictions
prediction_dates = df["Date"].iloc[split_idx:].reset_index(drop=True)
print("Predictions:", len(predicted_prices))
print("Actual values:", len(actual_prices))
print("Dates:", len(prediction_dates))
mae = mean_absolute_error(actual_prices, predicted_prices)
```

```
rmse = np.sqrt(mean_squared_error(actual_prices, predicted_prices))
# MAPE can be unstable when actual values are zero.
non_zero = actual_prices != 0
mape = np.mean(
    np.abs(
        (actual_prices[non_zero] - predicted_prices[non_zero])
        / actual_prices[non_zero]
    )
) * 100
print(f"MAE  : ${mae:.2f}")
print(f"RMSE : ${rmse:.2f}")
print(f"MAPE : {mape:.2f}%")
# Actual vs Predicted
plt.figure(figsize=(15, 6))
plt.plot(prediction_dates, actual_prices, label="Actual Price")
plt.plot(prediction_dates, predicted_prices, label="Predicted Price")
plt.title("AAPL: Actual vs Predicted Closing Price")
plt.xlabel("Date")
plt.ylabel("Closing Price (USD)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
# Zoom into the most recent 120 test observations
n_zoom = min(120, len(actual_prices))
plt.figure(figsize=(15, 6))
```

```
plt.plot(
    prediction_dates.iloc[-n_zoom:],
    actual_prices[-n_zoom:],
    label="Actual Price"
)
plt.plot(
    prediction_dates.iloc[-n_zoom:],
    predicted_prices[-n_zoom:],
    label="Predicted Price"
)
plt.title(f"AAPL: Recent {n_zoom} Test Observations")
plt.xlabel("Date")
plt.ylabel("Closing Price (USD)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
results = pd.DataFrame({
    "Date": prediction_dates,
    "Actual_Close": actual_prices,
    "Predicted_Close": predicted_prices
})
results["Absolute_Error"] = np.abs(
    results["Actual_Close"] - results["Predicted_Close"]
)
display(results.tail(15))
```

```
# Last 60 observed scaled prices
last_60 = scaler.transform(df[["Close"]].tail(LOOKBACK).values)
X_next = last_60.reshape(1, LOOKBACK, 1)
next_scaled = model.predict(X_next, verbose=0)
next_price = scaler.inverse_transform(next_scaled)[0, 0]
last_date = df["Date"].iloc[-1]
# Approximate next business day (actual market holidays are not considered here)
next_business_day = last_date + pd.tseries.offsets.BDay(1)
print(f"Last observed date : {last_date.date()}")
print(f"Last actual close   : ${df['Close'].iloc[-1]:.2f}")
print(f"Forecast date       : {next_business_day.date()}")
print(f"Forecasted close    : ${next_price:.2f}")
```

Output :



	Close Price
count	2766.000000
mean	108.644931
std	73.856228
min	22.584999
25%	39.080626
50%	91.204998
75%	170.830006
max	286.190002

	Date	Close
2756	2025-12-17	271.839996
2757	2025-12-18	272.190002
2758	2025-12-19	273.670013
2759	2025-12-22	270.970001
2760	2025-12-23	272.359985
2761	2025-12-24	273.809998
2762	2025-12-26	273.399994
2763	2025-12-29	273.760010
2764	2025-12-30	273.079987
2765	2025-12-31	271.859985

```
Training observations: 2212
Testing observations : 554
Training period: 2015-01-02 to 2023-10-16
Testing period : 2023-10-17 to 2025-12-31
```

```
Original training range: 22.585 to 196.45
Scaled training range  : 0.0 to 1.0000001
```

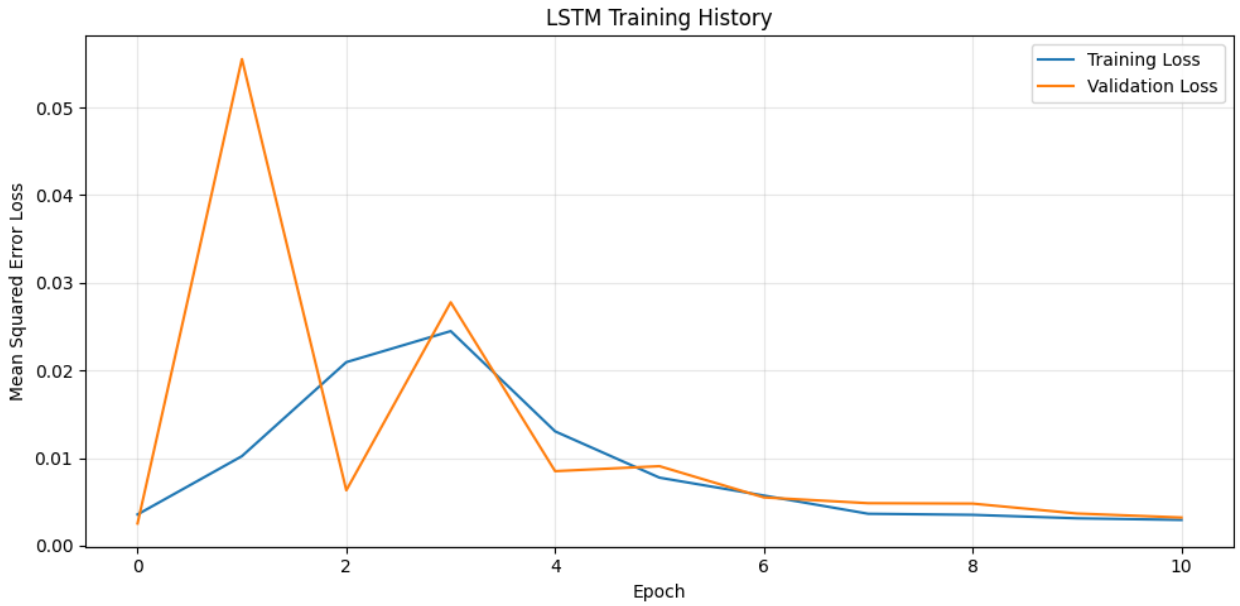
Model: "sequential"

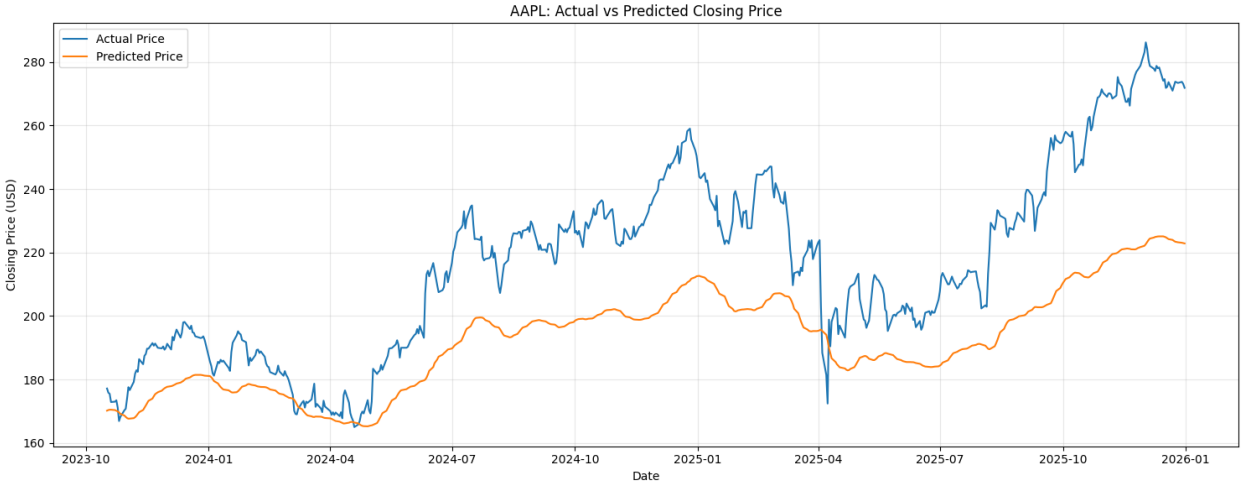
Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 64)	16,896
dropout (Dropout)	(None, 60, 64)	0
lstm_1 (LSTM)	(None, 32)	12,416
dropout_1 (Dropout)	(None, 32)	0
dense (Dense)	(None, 16)	528
dense_1 (Dense)	(None, 1)	17

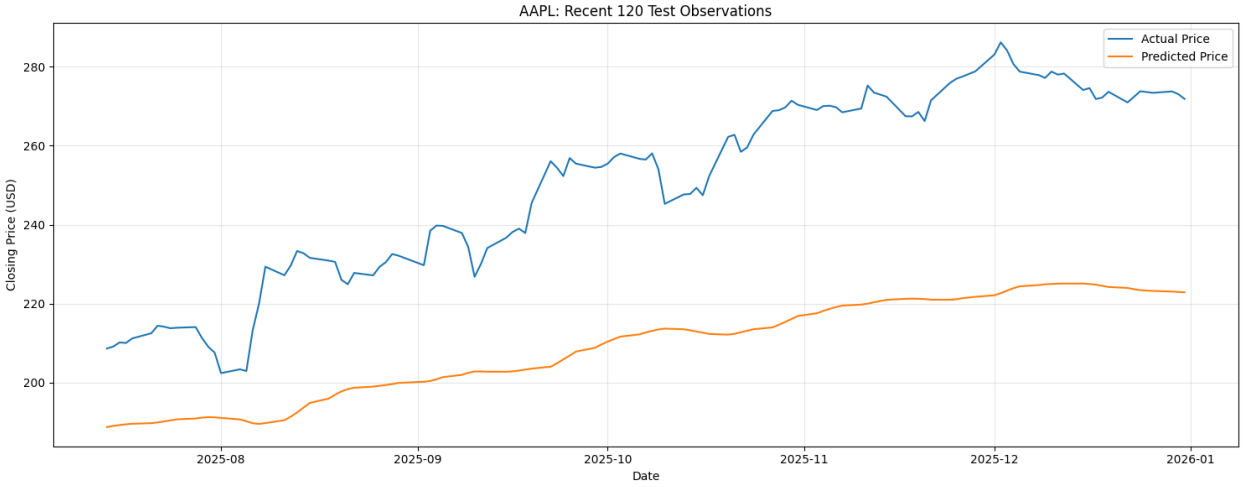
Total params: 29,857 (116.63 KB)

Trainable params: 29,857 (116.63 KB)

Non-trainable params: 0 (0.00 B)




AAPL: Actual vs Predicted Closing Price
Actual Price
Predicted Price
Closing Price (USD)
Date
160
180
200
220
240
260
280
2023-10
2024-01
2024-04
2024-07
2024-10
2025-01
2025-04
2025-07
2025-10
2026-01


AAPL: Recent 120 Test Observations
Actual Price
Predicted Price
Closing Price (USD)
Date
200
220
240
260
280
2025-08
2025-09
2025-10
2025-11
2025-12
2026-01

	Date	Actual_Close	Predicted_Close	Absolute_Error
539	2025-12-10	278.779999	224.980530	53.799469
540	2025-12-11	278.029999	225.051605	52.978394
541	2025-12-12	278.279999	225.075745	53.204254
542	2025-12-15	274.109985	225.081696	49.028290
543	2025-12-16	274.609985	224.950684	49.659302
544	2025-12-17	271.839996	224.780518	47.059479
545	2025-12-18	272.190002	224.508224	47.681778
546	2025-12-19	273.670013	224.213074	49.456940
547	2025-12-22	270.970001	223.960068	47.009933
548	2025-12-23	272.359985	223.657471	48.702515
549	2025-12-24	273.809998	223.397079	50.412918
550	2025-12-26	273.399994	223.210632	50.189362
551	2025-12-29	273.760010	223.059662	50.700348
552	2025-12-30	273.079987	222.952148	50.127838
553	2025-12-31	271.859985	222.854675	49.005310

```
Last observed date : 2025-12-31
Last actual close   : $271.86
Forecast date       : 2026-01-01
Forecasted close    : $222.74
```